Laboratorio N°3 b

"FUNCION POW (X^y) - EXPONENCIACION BINARIA LC3"

FACULTAD DE INGENIERÍA - UNAM

CARRERA:

Ingeniería en Computación

MATERIAS:

IC221 - Arquitectura de computadoras

DOCENTES:

ING. CARBALLO, Roberto Esteban

ING. BENITEZ, Aldo Javier

ING. SCHUSTER, Roberto Nicolás

ING. ROSSLER, Gustavo Omar

ING. DIAZ, Eduardo Sebastian

ING. FELTAN, Patricia Vanesa

SR. Giménez, Nahuel Joaquín

SR. ZUK, Joaquín

ALUMNOS:

SARI FERNÁNDEZ, Maximiliano Gastón

SOLIS, Carlos Esteban

VILLALBA, Agustin

GRUPO: 4

ÍNDICE

Introducción 4
Fundamento Matemático 4
1. Implementacion en lenguaje de alto nivel (C) 5
1.1. Codigo. 5
1.2. Conclusion 7
2. Implementacion en Assembly para la LC-3 8
2.1. Introduccion 8
2.2. Funcionamiento del programa 8
2.3. Resultados 11
2.3.1. Manejo de base negativa y restricción de exponente negativo 11
2.3.2. Rango de datos aceptado 11
2.3.3. Formato de punto fijo y enteros 12
2.3.4. Rango de la salida 12
2.3.5. Precisión mínima y máxima 12
CONCLUSIÓN 14
REFERENCIAS BIBLIOGRÁFICAS 14

Introducción

La idea central es calcular una potencia sin realizar una multiplicación por cada unidad del exponente, aprovechando que podemos transformar una potencia en otra equivalente cuyo exponente sea aproximadamente la mitad.

Fundamento Matemático

Fundamento 1: Multiplicar potencias de la misma base

Cuando multiplicamos dos potencias que tienen la misma base, sumamos sus exponentes:

$a^m \cdot a^n = a^{m+n}$ | Ejemplo: $2^3 \cdot 2^2 = 2^{3+2} = 2^5 = 32$

Fundamento 2: Potencia de una potencia

Si elevamos una potencia a otro exponente, multiplicamos los exponentes:

$(a^m)^n = a^{m \cdot n}$ | Ejemplo: $(2^2)^3 = 2^{2 \cdot 3} = 2^6 = 64$

En particular, si elevamos una base al cuadrado: $(a^2)^k = a^{2k}$. Por lo tanto, podemos duplicar el exponente de una potencia simplemente elevando al cuadrado su base.

Fundamento 3: Separar un exponente par

Supongamos que queremos calcular a^8:

$a^8 = a^4 \cdot a^4 = (a^2)^4 = ((a^2)^2)^2$

En términos de cálculo, podemos obtener a^2, después a^4 y finalmente a^8 usando solo tres multiplicaciones:

- $a^2 = a \cdot a$
- $a^4 = a^2 \cdot a^2$
- $a^8 = a^4 \cdot a^4$

Fundamento 4: Exponente es impar

Si es impar, podemos separar un factor de la base:

$a^n = a \cdot a^{n-1} \quad \Rightarrow \quad a^5 = a \cdot a^4 = a \cdot (a^2)^2$

Así, cuando el exponente es impar, guardamos un factor en el resultado y luego reducimos el exponente.

1. Implementacion en lenguaje de alto nivel (C)

1.1. Codigo.

En este diseño enemos tres variables principales:

- base: la base actual que estamos utilizando.
- exp: la parte del exponente que todavía tenemos que procesar.
- resultado: el producto de los factores que ya hemos incorporado (inicializado en 1).

Procedimiento iterativo:

1. Comprobar si el exponente es impar (exp % 2 == 1): Incorporamos la base actual al resultado.
2. Reducir el exponente a la mitad (exp = exp / 2): Realizamos una división entera.
3. Elevar la base al cuadrado: Si todavía quedan pasos (exp > 0), calculamos base * base.

```c
int pow_binaria(int x, int y)
{
    int resultado = 1;
    int base = x;
    int exponente = y;

    while (exponente > 0)
    {
        if (exponente % 2 == 1)
        {
            resultado = multiplicar(resultado, base);
        }

        base = multiplicar(base, base);
        exponente = exponente / 2;
    }

    return resultado;
}
```

***Fig. 1:** Bucle Principal Potenciación Binaria.*

A continuación se muestra una tabla con el comportamiento del exponente al dividirse por 2:

***Tabla. 1:** Comportamiento exp/2.*

Valor anterior	Operación	Nuevo valor
13	13 / 2	6
6	6 / 2	3
3	3 / 2	1
1	1 / 2	0

A continuación, se presenta una tabla con las iteraciones que realiza el programa para el ejemplo:

***Tabla. 2:** Tabla de iteraciones.*

Iteración	Base inicio	Exponente	Acción sobre el resultado	Resultado
1	2	13	Multiplicar por 2 (1 · 2)	2
2	4	6	Ninguna (exponente par)	2
3	16	3	Multiplicar por 16 (2 · 16)	32
4	256	1	Multiplicar por 256 (32 · 256)	8192

La cantidad de iteraciones del ciclo principal crece de forma logarítmica $O(\log n)$: $I(n) = \lfloor \log_2(n) \rfloor + 1$.
A continuación se presenta una tabla con iteración según el exponente:

***Tabla. 3:** Tabla de iteraciones teniendo en cuenta el exponente.*

Exponente (n)	Iteraciones del while
1	1
5	3
8	4
13	4
100	7
1.000	10
1.000.000	20

A continuación se presenta una tabla con el procesamiento de bits:

Tabla. 4: *Tabla de procesamiento de bits del exponente.*

Bit	Potencia de 2	Valor decimal
1	2^3	8
1	2^2	4
0	2^1	0
1	2^0	1

1.2. Conclusion

El programa procesa los bits de derecha a izquierda: [1, 0, 1, 1]

1. exp % 2: Obtiene el bit menos significativo.

2. exp / 2: Elimina ese bit desplazando hacia la derecha.

3. base = multiplicar(base, base): Eleva al cuadrado para preparar la potencia equivalente al siguiente bit.

2. Implementacion en Assembly para la LC-3

2.1. Introduccion

El programa implementa una función de exponenciación binaria rápida (también conocida como exponenciación por cuadrados sucesivos) en la arquitectura de la LC-3. Al no contar con instrucciones nativas de multiplicación ni de división por hardware, se limitó a sumar, restar de manera sucesiva. El programa calcula la potencia $Base^{Exponente}$ (por ejemplo, $(-2)^{13}$) manejando correctamente bases negativas, paridad de exponentes y optimizando el rendimiento mediante el algoritmo de exponenciación rápida, reduciendo la complejidad temporal de O(E) a $O(\log_2 E)$.

2.2. Funcionamiento del programa

Para resolver la exponenciación en la LC-3 sin instrucciones complejas, el programa se divide en etapas lógicas:

- **Inicialización de Variables:** Se cargan los valores iniciales en los registros (Base en R1, Exponente en R2, Acumulador en R3 = 1).

```
; --- 1. INICIALIZACIÓN DE VARIABLES EN REGISTROS ---
AND R1, R1, #0
ADD R1, R1, #-2          ; R1 = -2 (Base)

AND R2, R2, #0
ADD R2, R2, #13          ; R2 = 13 (Exponente)

AND R3, R3, #0
ADD R3, R3, #1           ; R3 = 1 (Acumulador inicial)
```

***Fig 2.** Variables iniciales*

- **Gestión de Signos y Paridad:** Se evalúa si la base es negativa y si el exponente es impar (R2 & 1). Si la base es negativa y el exponente es impar, el resultado final será negativo; de lo contrario, será positivo. Se opera temporalmente con el valor absoluto de la base.

```
; --- 2. CONFIGURACIÓN DE SIGNOS Y PARIDAD EN REGISTROS ---
 AND R6, R6, #0
 AND R0, R2, #1
 ADD R6, R6, R0          ; R6 (TMP_MEM) = R2 & 1 (guarda si el exponente es impar)

 AND R5, R5, #0          ; R5 (TMP_SIGN) = 0 por defecto (positivo)
 ADD R1, R1, #0
 BRzp LOOP_W             ; Si es positivo o cero, salta directo al bucle principal

 ADD R5, R5, #1          ; Si la base es negativa, R5 = 1
 NOT R1, R1
 ADD R1, R1, #1          ; Base positiva temporalmente para operar
```

Fig 3. Gestion de signos

- **Bucle Principal de Exponenciación Rápida (LOOP_W):**
  - En cada iteración se verifica si el exponente es menor o igual a cero.
  - Se comprueba si el bit menos significativo del exponente es 1 (AND R0, R2, #1). Si es 1, se multiplica el acumulador actual por la base actual mediante sumas sucesivas (M_LOOP1).

```
        ; Multiplicación por sumas sucesivas (R3 = R3 * R1)
        AND R0, R0, #0
        ADD R4, R1, #0
M_LOOP1
        ADD R4, R4, #0
        BRnz M_END1
        ADD R0, R0, R3
        ADD R4, R4, #-1
        BR M_LOOP1
M_END1
```

Fig 4. Multiplicacion con suma sucesiva

  - Se divide el exponente entre 2 simulando un desplazamiento mediante restas sucesivas de 2 (D_LOOP).

```
        ; División del exponente entre 2 (Shift simulado por restas de 2)
L_SHIFT
        AND R0, R0, #0
        ADD R4, R2, #0
        AND R2, R2, #0
D_LOOP
        ADD R4, R4, #-2
        BRn D_END
        ADD R2, R2, #1
        BR D_LOOP
D_END
```

Fig 5. Division con suma sucesiva

- Se eleva la base al cuadrado mediante multiplicaciones sucesivas (M_LOOP2) para prepararla para la siguiente potencia de dos.

```
        ; Multiplicación de cuadrados sucesivos (R1 = R1 * R1)
        AND R0, R0, #0
        ADD R4, R1, #0
M_LOOP2
        ADD R4, R4, #0
        BRnz M_END2
        ADD R0, R0, R1
        ADD R4, R4, #-1
        BR M_LOOP2
M_END2
        ADD R1, R0, #0

        BR LOOP_W
```

Fig 6. Cuadrado de base

- **Procesamiento Final y Ajuste de Signos:** Al finalizar el bucle, si se determinó que el resultado debía ser negativo por la combinación de base negativa y exponente impar, se aplica complemento a dos (NOT y +1) sobre el acumulador.

```
        ; --- 4. PROCESAMIENTO FINAL Y AJUSTE DE SIGNOS ---
POST_PROC
        ADD R5, R5, #0
        BRz ST_RES              ; Si la base era positiva, guarda directo
        ADD R6, R6, #0
        BRz ST_RES              ; Si el exponente era par, guarda directo

        ; Si base negativa y exponente impar, el resultado final es negativo
        NOT R3, R3
        ADD R3, R3, #1
```

Fig 7. Procesamiento de ajuste de signo

- **Acceso Único a Memoria:** El resultado final se almacena en la celda de memoria FINAL_RES y se detiene la ejecución con HALT.

```
        ; --- 5. ÚNICO ACCESO A MEMORIA ---
ST_RES
        ST R3, FINAL_RES        ; Guarda el resultado final en memoria RAM
        HALT                    ; Fin del programa

; --- DATOS ---
FINAL_RES   .BLKW #1

        .END
```

Fig 8. Guardado de datos

2.3. Resultados

2.3.1. Manejo de base negativa y restricción de exponente negativo

El programa está diseñado para operar con bases negativas y exponentes enteros no negativos (E>=0):

- **Base negativa**: Si la base introducida en R1 es negativa, el algoritmo la detecta mediante la instrucción condicional BRzp, activa la bandera de signo (R5 = 1) y obtiene su valor absoluto mediante complemento a dos (NOT R1, R1 seguido de ADD R1, R1, #1). Esto permite ejecutar los bucles de multiplicación por sumas sucesivas operando temporalmente con magnitudes positivas.
- **Paridad del exponente**: La paridad del exponente original se evalúa al inicio con AND R0, R2, #1 y se almacena en la bandera R6.
- **Ajuste final de signo**: En POST_PROC, el resultado almacenado en R3 solo se invierte a negativo mediante complemento a dos si la base original era negativa (R5 == 1) y el exponente era impar (R6 == 1). Si la base era positiva o el exponente era par, el resultado se conserva positivo.
- **Exponente negativo**: El programa no admite exponentes negativos (E < 0). Dado que la LC-3 opera con aritmética entera de 16 bits (Q15.0), las potencias con exponentes negativos resultan en fracciones reales menores a 1 (por ejemplo, $2^{(-3)} = 0.125$), las cuales en aritmética entera se truncarían a cero. Si R2 ingresara con un valor negativo, la instrucción condicional BRnz al inicio de LOOP_W saltaría inmediatamente a POST_PROC, devolviendo el acumulador inicial (R3 = 1) sin evaluar el bucle.

2.3.2. Rango de datos aceptado

El programa acepta como entrada cualquier entero de 16 bits con signo expresado en complemento a dos en la arquitectura LC-3, dentro del rango estricto de [-32,768 ; +32,767] ; $[(-2)^{15} ; 2^{15} - 1]$.

- **Base (B):** Puede tomar cualquier valor entero positivo o negativo dentro del rango.

- **Exponente (E):** Únicamente se aceptan valores enteros positivos o cero [0 ; 32,767].

2.3.3. Formato de punto fijo y enteros

Tanto las entradas (base en R1 y exponente en R2) como las operaciones intermedias y el resultado final se manejan en formato Q15.0 (enteros puros de 16 bits con signo en complemento a dos).

- **Resolución de los datos:** Al operar en Q15.0, el paso mínimo o resolución es de $2^0 = 1$ entero.

- **Representación en memoria:** Un valor de -8192 se almacena exactamente como xE000 en la palabra de memoria de 16 bits designada. No se requiere ningún reescalado o desplazamiento de bits (*normalization shift*) al no incluir parte fraccionaria.

2.3.4. Rango de la salida

El rango de la salida abarca la totalidad del espacio de direccionamiento de palabras con signo de la LC-3, desde [-32,768 ; +32,767].

En la verificación del caso de prueba con la entrada B = -2 y E = 13, la salida FINAL_RES toma exactamente el valor -8192 (xE000 en hexadecimal). El programa garantiza la preservación matemática exacta de las propiedades del signo: una base negativa elevada a un exponente impar produce un resultado estrictamente negativo dentro del rango permitido.

2.3.5. Precisión mínima y máxima

Dado que la exponenciación binaria sobre enteros opera mediante sumas y multiplicaciones exactas, no existe error numérico de truncamiento ni de redondeo.

- **Error de aproximación: 0 LSB** (0% de error). El resultado es exacto bit a bit respecto a la especificación matemática B^E, siempre que no ocurra un desbordamiento.
- **Precisión:** Se alcanza el 100% de coincidencia exacta.

***Tabla. 5:** Tabla de errores.*

Medida	Valor
Error mínimo	0 LSB (Coincidencia exacta bit a bit)
Error máximo	0 LSB (Sin error de truncamiento ni aproximación)
Error medio	0 LSB
Coincidencia en entradas válidas	100%

***Tabla. 6:** Tabla de valores de control.*

Base (R_1)	Exponente (R_2)	Resultado en FINAL_RES	Valor Decimal Exacto	Estado
2	**0**	**x0001**	**1**	**Correcto**
-2	**3**	**xFFF8**	**-8**	**Correcto**
3	**4**	**x0051**	**81**	**Correcto**
-2	**13**	**xE000**	**-8192**	**Caso principal verificado**
-1	**15**	**xFFFF**	**-1**	**Correcto**

CONCLUSIÓN

- Se implementó con éxito el algoritmo de **exponenciación binaria por cuadrados sucesivos** en la arquitectura LC-3, permitiendo calcular potencias de la forma B^E mediante una combinación eficiente de sumas repetitivas y desplazamientos simulados.

- La incorporación del análisis de paridad del exponente y el control de signo en la base mediante complemento a dos (NOT y +1) permitió extender la funcionalidad para operar correctamente con **bases negativas**, respetando las leyes algebraicas del signo y restringiendo los exponentes a valores enteros no negativos (E >= 0).

- El uso del método de exponenciación rápida redujo la complejidad temporal a nivel del control del exponente de un orden lineal O(E) a un orden logarítmico **$O(\log_2 E)$**, ejecutando únicamente 4 iteraciones principales para el caso de prueba $(-2)^{13}$.

- La estrategia de diseño orientada a **mapeo exclusivo en registros** (R_0 a R_6) minimizó la latencia de ejecución al eliminar lecturas y escrituras intermedias a la memoria RAM, limitando el acceso a memoria a una única instrucción de almacenamiento al finalizar el programa (ST R3, FINAL_RES).

- Operando en formato **Q15.0** (enteros con signo de 16 bits), el algoritmo alcanzó **exactitud matemática absoluta (0 LSB de error)**, obteniendo de forma precisa el resultado final de ejemplo -8192 (xE000) sin degradación por redondeo ni truncamiento.

.

REFERENCIAS BIBLIOGRÁFICAS

[1] S. Harris and D. Harris, Digital design and computer architecture: arm edition: Morgan Kaufmann, 2015.

[2] Y. N. Patt and S. J. Patel, Introduction to Computing Systems: From Bits & Gates to C & Beyond, 2nd ed.: McGraw-Hill, 2004.

[3] J.-M. Muller, Elementary Functions: Algorithms and Implementation, 3rd ed.: Birkhäuser, 2016.

[5] L. N. Trefethen, Approximation Theory and Approximation Practice: SIAM, 2013.